

Struktur Data dan Keamanan Kode: Mengelola Array, Method, dan Robust Error Handling di Go

Durasi: 3 Jam (180 Menit) **Level:** Intermediate (Menengah)

Sebagai Backend Engineer Go, Anda harus memahami bagaimana Go mengelola data kelompok di memori, merancang arsitektur kode menggunakan Method yang modular, serta menguasai filosofi penanganan kesalahan (*Error Handling*) standar industri.

Modul ini akan membedah secara mendalam struktur internal Array dan Slice, cara kerja Penerima Method (*Method Receivers*), serta mekanisme *Error Handling* tingkat lanjut seperti *Sentinel Error*, *Custom Error Type*, *Error Wrapping*, dan penggunaan `defer-panic-recover`.

Sesi 1: Koleksi Data dengan Array dan Slice (45 Menit)

Meskipun terlihat mirip dengan list/array di bahasa lain, Go memiliki arsitektur manajemen memori yang unik untuk struktur data kelompok.

1. Array (Fixed Size)

Array di Go berukuran tetap (*fixed size*) yang ditentukan saat kompilasi. Ukuran array merupakan bagian dari tipe datanya.

```
var prices [3]float64
prices[0] = 15000.0
prices[1] = 25000.5
// prices[3] = 10000.0 // Compile Error: out of bounds

// Ukuran array tidak bisa diubah. [3]int dan [5]int adalah tipe data yang berbeda
```

2. Slice (Dynamic Size & Under the Hood)

Slice adalah abstraksi fleksibel di atas array. Slice sendiri tidak menyimpan data secara fisik; ia bekerja sebagai referensi (*window*) ke **Backing Array** yang mendasarinya.

Secara internal, sebuah Slice adalah struct kecil dengan 3 komponen (24 bytes pada arsitektur 64-bit):

1. **Pointer**: Alamat memori elemen pertama di backing array yang dirujuk.
2. **Length (len)**: Jumlah elemen yang saat ini ada di dalam slice.
3. **Capacity (cap)**: Kapasitas maksimum backing array sebelum alokasi ulang memori diperlukan.

```
package main

import "fmt"

func main() {
    // Membuat slice dengan make: len = 3, cap = 5
    scores := make([]int, 3, 5)
    scores[0], scores[1], scores[2] = 85, 90, 95

    fmt.Printf("Scores: %v | Len: %d | Cap: %d\n", scores, len(scores), cap(scores))

    // Menambahkan data melebihi len (3), tapi masih di bawah cap (5)
    scores = append(scores, 100)
    fmt.Printf("Setelah Append: %v | Len: %d | Cap: %d\n", scores, len(scores), cap(scores))
}
```

Cara Kerja `append` dan Alokasi Memori

Jika kita menambahkan data menggunakan `append` dan panjang slice melebihi kapasitasnya (`len > cap`), Go akan otomatis:

1. Membuat backing array baru di memori dengan ukuran yang lebih besar (biasanya 2 kali lipat dari kapasitas lama).
2. Menyalin (*copy*) semua data lama ke backing array baru.
3. Mengarahkan pointer slice ke backing array yang baru.
4. Menghapus backing array lama melalui *Garbage Collector*.

WARNING

Performance Gotcha: Alokasi ulang backing array yang berulang-ulang akibat pemanggilan `append` terus-menerus bisa menurunkan performa aplikasi secara signifikan. Jika Anda sudah tahu perkiraan jumlah elemen maksimum yang akan ditampung, selalu inisialisasi slice menggunakan `make([]T, len, cap)` untuk menghindari realokasi memori berlebih.

Pengkopian Slice yang Aman (`copy`)

Jika Anda meng-assign slice `b := a`, kedua slice akan menunjuk ke backing array yang sama. Modifikasi elemen di `b` akan memengaruhi `a`. Gunakan fungsi `copy()` untuk menduplikasi data ke memori terpisah:

```
src := []int{1, 2, 3}
dest := make([]int, len(src)) // Harus inisialisasi kapasitas dest terlebih dahulu
copy(dest, src) // Menyalin elemen dari src ke dest secara aman
```

Sesi 2: Method dalam Go (45 Menit)

Go tidak memiliki konsep kelas (*Class*), namun Anda dapat mendeklarasikan **Method** pada `struct` atau tipe data kustom apa pun. Method adalah fungsi yang dideklarasikan dengan argumen penerima (*receiver*) khusus.

1. Value Receiver vs Pointer Receiver

Perbedaan receiver menentukan bagaimana data dilewatkan ke method:

Kriteria	Value Receiver (<code>t T</code>)	Pointer Receiver (<code>t *T</code>)
Pelewatan Memori	Menyalin (<i>copy</i>) seluruh data struct ke memori baru.	Melewatkan alamat memori (pointer) asli.
Modifikasi Data	Tidak bisa mengubah data struct asal (hanya <i>read-only</i>).	Dapat mengubah data struct asal secara permanen.
Penggunaan Ideal	Struct kecil (seperti data koordinat), hemat penyalinan.	Struct besar (mengurangi overhead copy memori) atau butuh ubah data.

```

package main

import "fmt"

type Product struct {
    Name    string
    Price   float64
}

// Value Receiver: Hanya membaca data
func (p Product) Display() {
    fmt.Printf("Produk: %s | Harga: Rp %.2f\n", p.Name, p.Price)
}

// Pointer Receiver: Mengubah nilai asli di memori
func (p *Product) ApplyDiscount(discount float64) {
    p.Price -= discount
}

```

2. Method pada Tipe Kustom Non-Struct

Anda bisa mendefinisikan method pada tipe data primitif yang telah di-alias:

```

type OrderStatus int

const (
    Pending OrderStatus = iota
    Processed
    Shipped
)

// Menambahkan method pada tipe alias int
func (s OrderStatus) String() string {
    return [...]string{"PENDING", "PROCESSED", "SHIPPED"}[s]
}

```

3. Nil Receivers (Fitur Unik Go)

Di Go, sebuah method dapat dipanggil pada pointer yang bernilai `nil` tanpa menyebabkan panic *Null Pointer Exception*, asalkan logika internal method menangani pengecekan tersebut:

```

func (p *Product) GetNameSafely() string {
    if p == nil {
        return "Produk Tidak Tersedia"
    }
    return p.Name
}

```

Sesi 3: Robust Error Handling (45 Menit)

Filosofi Go menganggap error bukan sebagai pengecualian luar biasa (*exception*), melainkan sebagai **nilai biasa** (*value*) yang dikembalikan fungsi di urutan terakhir.

1. Sentinel Errors (errors.Is)

Sentinel error adalah error tetap yang didefinisikan sebagai variabel tingkat package untuk memudahkan pengecekan logika di bagian sistem lain.

```
package main

import (
    "errors"
    "fmt"
)

// Definisi Sentinel Errors (Gunakan prefix Err...)
var (
    ErrStockEmpty    = errors.New("inventory: stok barang habis")
    ErrItemNotFound = errors.New("inventory: barang tidak terdaftar")
)

func CheckInventory(stock int) error {
    if stock == 0 {
        return ErrStockEmpty
    }
    return nil
}

func main() {
    err := CheckInventory(0)
    // Pengecekan aman menggunakan errors.Is
    if errors.Is(err, ErrStockEmpty) {
        fmt.Println("Gagal: Hubungi vendor untuk re-stock!")
    }
}
```

2. Custom Error Struct (errors.As)

Jika Anda membutuhkan metadata tambahan pada error (seperti waktu error terjadi atau id transaksi), buatlah custom struct yang mengimplementasikan interface `error` bawaan Go dengan menulis method `Error() string` :

```

type PaymentError struct {
    TransactionID string
    Code          int
    Message       string
}

// Implementasi interface error
func (e *PaymentError) Error() string {
    return fmt.Sprintf("[Payment Error %d] Transaksi %s Gagal: %s", e.Code, e.TransactionID, e.Message)
}

```

Untuk menguji dan mengekstrak tipe custom error ini, gunakan `errors.As` :

```

var payErr *PaymentError
if errors.As(err, &payErr) {
    fmt.Printf("Gagal Bayar Kode %d pada ID Transaksi %s\n", payErr.Code, payErr.TransactionID)
}

```

3. Error Wrapping (%w)

Saat melewati error ke atas (*call stack*), kita sering ingin menambahkan konteks baru tanpa kehilangan identitas error aslinya. Gunakan parameter `%w` pada `fmt.Errorf` :

```

func DatabaseQuery() error {
    return ErrItemNotFound // Sentinel error
}

func GetProduct() error {
    err := DatabaseQuery()
    if err != nil {
        // Wrapping error dengan %w
        return fmt.Errorf("gagal memuat produk: %w", err)
    }
    return nil
}

func main() {
    err := GetProduct()
    fmt.Println(err) // Output: gagal memuat produk: inventory: barang tidak terdaftar

    // errors.Is tetap bisa mendeteksi Sentinel Error di dalam rantai wrapping!
    if errors.Is(err, ErrItemNotFound) {
        fmt.Println("Detail: Ternyata barang memang tidak ada di database.")
    }
}

```

4. Defer, Panic, dan Recover

- **defer** : Menunda eksekusi suatu fungsi hingga fungsi luar selesai dieksekusi (sangat berguna untuk merapikan koneksi DB atau file).
- **panic** : Menghentikan alur normal program secara paksa saat terjadi kesalahan fatal yang tidak dapat dipulihkan.
- **recover** : Fungsi bawaan untuk menangkap *panic* dan mencegah program crash. Wajib dipanggil di dalam blok `defer`.

```
package main

import "fmt"

func SafeExecution() {
    defer func() {
        if r := recover(); r != nil {
            fmt.Println("Sistem berhasil pulih dari panic:", r)
        }
    }()

    fmt.Println("Memulai proses...")
    panic("koneksi database terputus total!") // Memicu panic
    fmt.Println("Baris ini tidak akan pernah dieksekusi.")
}

func main() {
    SafeExecution()
    fmt.Println("Program tetap berjalan aman setelah pemulihan.")
}
```

Sesi 4: Praktikum Mandiri: "Sistem Manajemen Inventaris & Penjualan" (45 Menit)

Mari kita gabungkan konsep Slice dinamis, Method Receivers, dan penanganan kesalahan standar industri ke dalam sistem manajemen inventaris toko e-commerce.

Implementasi Kode (`main.go`)

```
package main

import (
    "errors"
    "fmt"
)

// 1. Definisikan Sentinel Errors untuk validasi data
var (
    ErrEmptyName      = errors.New("invalid: nama produk tidak boleh kosong")
    ErrNegativeVal     = errors.New("invalid: harga atau stok tidak boleh bernilai nega
    ErrInsufficient   = errors.New("checkout: stok barang tidak mencukupi")
    ErrNotFound        = errors.New("checkout: barang tidak ditemukan di sistem")
)

// 2. Struct Model
type Item struct {
    Name  string
    Price float64
    Stock int
}

type StoreInventory struct {
    Items []Item
}

// 3. Method untuk validasi & tambah barang (Pointer Receiver)
func (inv *StoreInventory) AddProduct(name string, price float64, stock int) error {
    if name == "" {
        return ErrEmptyName
    }
    if price < 0 || stock < 0 {
        return ErrNegativeVal
    }

    newItem := Item{Name: name, Price: price, Stock: stock}
    inv.Items = append(inv.Items, newItem)
    return nil
}

// 4. Method untuk pemrosesan penjualan (Pointer Receiver)
func (inv *StoreInventory) SellProduct(name string, qty int) (float64, error) {
    for i, item := range inv.Items {
        if item.Name == name {
            if item.Stock < qty {
                // Menggunakan Error Wrapping untuk menyisipkan info sto
                return 0, fmt.Errorf("%w (Stok Tersedia: %d, Permintaan: %d)", ErrInsufficient, item.Stock, qty)
            }

            // Kurangi stok pada backing array asli
        }
    }
}
```



```

        inv.Items[i].Stock -= qty
        totalIncome := item.Price * float64(qty)
        return totalIncome, nil
    }
}

// Mengembalikan Sentinel Error
return 0, fmt.Errorf("%w: produk '%s'", ErrNotFound, name)
}

func main() {
    store := StoreInventory{}

    // Menambahkan produk secara dinamis
    _ = store.AddProduct("Laptop Gaming", 15000000.0, 5)
    _ = store.AddProduct("Mouse Wireless", 350000.0, 15)

    fmt.Println("=====")
    fmt.Println("          SIMULASI TRANSAKSI E-COMMERCE          ")
    fmt.Println("=====")

    // Uji 1: Penjualan Berhasil
    if total, err := store.SellProduct("Mouse Wireless", 5); err != nil {
        fmt.Println("Gagal Transaksi:", err)
    } else {
        fmt.Printf("[SUKSES] Penjualan Mouse Wireless x5. Total: Rp %.2f\n", tot
    }

    // Uji 2: Penjualan Gagal (Stok Kurang) - Menguji Is dengan error wrapped
    _, errSales := store.SellProduct("Laptop Gaming", 10)
    if errSales != nil {
        fmt.Println("[GAGAL]", errSales)
        if errors.Is(errSales, ErrInsufficient) {
            fmt.Println("Tindakan: Memicu notifikasi otomatis ke tim gudang
        }
    }

    // Uji 3: Penjualan Gagal (Barang Tidak Ditemukan)
    _, errNotFound := store.SellProduct("Keyboard RGB", 1)
    if errNotFound != nil {
        fmt.Println("[GAGAL]", errNotFound)
    }
}

```

Sesi 5: Soal Latihan Praktik E-Commerce (Mandiri)

Selesaikan dua latihan pemrograman e-commerce tingkat lanjut di bawah ini untuk memperkuat pemahaman Anda mengenai Method, Custom Error, dan Error Wrapping.

Latihan 1: Sistem Keranjang Belanja dengan Kode Kupon & Sentinel Error

Deskripsi Soal: Buatlah sebuah sistem pengelolaan keranjang belanja e-commerce yang melayani penambahan barang dan kalkulasi tagihan menggunakan diskon kupon belanja.

Kriteria Implementasi:

1. Buat struct `CartItem` (berisi nama produk, harga satuan, dan kuantitas belanja) dan struct `ShoppingCart` (berisi daftar slice `[]CartItem`).
 2. Definisikan tiga buah **Sentinel Error**:
 - `ErrQtyInvalid` : Error jika kuantitas belanja kurang dari atau sama dengan 0.
 - `ErrInvalidCoupon` : Error jika kuantitas atau nominal diskon tidak cocok.
 - `ErrCartEmpty` : Error jika kalkulasi total dilakukan pada keranjang kosong.
 3. Implementasikan method-method berikut pada `ShoppingCart` :
 - `AddItem(item CartItem) error` : Memasukkan barang baru ke keranjang. Lakukan pengecekan validasi kuantitas. Jika nama produk sudah ada di keranjang, akumulasi saja jumlah belanjanya (tidak membuat item baru).
 - `ApplyDiscountCoupon(couponCode string) (float64, error)` : Menghitung persentase diskon berdasarkan kode kupon. Hanya mendukung kupon `"DISKON50"` (potongan 50%) dan `"CASHBACK10"` (potongan 10%). Jika kode kupon lain dimasukkan, kembalikan `ErrInvalidCoupon`.
 - `CalculateTotal(discountRate float64) (float64, error)` : Menghitung total belanja akhir setelah dipotong diskon. Jika keranjang kosong, kembalikan `ErrCartEmpty`.
 4. Di dalam fungsi `main()`, lakukan simulasi belanja lengkap: menambah beberapa item (termasuk produk bermasalah dengan qty negatif), mengklaim kupon valid dan tidak valid, dan mencetak invoice belanja yang rapi menggunakan `fmt.Printf`.
-

Latihan 2: Sistem Validasi Merchant E-Commerce dengan Custom Error Struct & errors.As

Deskripsi Soal: Dalam platform marketplace multi-merchant, setiap merchant harus melewati tahap registrasi produk yang ketat agar tidak ada produk dengan data yang merusak pasar (seperti harga tidak wajar atau nama produk spamming).

Kriteria Implementasi:

1. Buat struct `Product` (ID, Nama, Kategori, Harga, dan Stok).
2. Buat custom error struct bernama `MerchantValidationError` yang menyimpan informasi lengkap mengenai:
 - `MerchantID string`
 - `ProductID string`
 - `ViolationType string` (Jenis Pelanggaran, contoh: `"HARGA_TIDAK_WAJAR"` atau `"STOK_KOSONG"`)

- `Message string`
3. Buat struct `MarketplacePlatform` yang menampung daftar produk yang berhasil terverifikasi.
 4. Implementasikan method `RegisterMerchantProduct(merchantID string, prod Product) error` pada `MarketplacePlatform`. Lakukan validasi ketat:
 - Jika harga produk di bawah Rp 10.000 (tidak wajar), kembalikan error bertipe `MerchantValidationError` dengan violation type `"HARGA_TIDAK_WAJAR"`.
 - Jika stok produk bernilai 0 saat registrasi, kembalikan error bertipe `MerchantValidationError` dengan violation type `"STOK_KOSONG"`.
 - Jika data valid, masukkan ke daftar produk terverifikasi.
 5. Di dalam fungsi `main()`, lakukan simulasi registrasi produk oleh beberapa merchant. Jika registrasi gagal:
 - Tangkap error-nya, kemudian gunakan `errors.As` untuk mendeteksi apakah kegagalan tersebut disebabkan oleh pelanggaran aturan merchant (`MerchantValidationError`).
 - Jika cocok, ekstrak data `MerchantID` dan `ViolationType` untuk dicetak sebagai log laporan kepatuhan sistem keamanan marketplace. Gunakan konsep **Error Wrapping** agar pesan error aslinya tetap terjaga saat dialirkan ke atas.